

- Document number: P1253R0
- Date: 2019-01-21
- Author: Steve Downey sdowney2@bloomberg.net
- Audience: SG16, LEWG, LEWGI, EWG, EWGI, WG21

Abstract: Guidelines for when a WG21 proposal should be reviewed by SG16, the text and Unicode study group.

Introduction

This paper provides some guidelines for when WG21 papers should be forwarded to study group 16 for review. The focus of study group 16 is text processing, with a specific focus on Unicode. Study group 16 will also review papers for issues with text encoding, text formatting, and IO.

Unicode Facilities

Any proposal that implements a general purpose Unicode text type, a view on Unicode text, or implements any of the Unicode standard facilities or algorithms should of course be forwarded to SG16. SG16 is currently reviewing proposals for `std::text` and `std::text_view`, so anything with those names should also be sent to SG16.

Any proposal that mentions Unicode may be sent for review, if just to get clarification of what is meant in context. We currently live in a multi-character set and encoding world, and in general it is difficult to require or specify that general text follows a particular encoding. If existing external standards, such as XML or JSON, require Unicode or a particular encoding, following those standards doesn't need particular review from SG16.

Using existing language and library facilities does not require review. For example, using `std::string`, `std::string_view`, etc. An exception would be using `char16_t`, `char32_t`, or `char8_t`, but only because those imply, or should imply, Unicode text.

Char{8,16,32}_t overloads or specializations

Any specific overload or specialization using the types `char16_t`, `char32_t`, or `char8_t` should be reviewed by Study Group 16. They are intended for Unicode text processing, and are therefore within the remit of SG16.

If the types are merely mentioned because they are trivial types, and there is no specialized behavior because of the type, SG16 does not need to review.

Text Encoding

Any proposal that transcodes text from host, source, execution, or other text encoding, to any of the Unicode text encodings, such as UTF-8, should be sent to Study Group 16. Any proposal that states that text is encoded in a particular specified encoding, such as UTF-18, or CP-1252, should be sent to Study Group 16, where the group can make recommendations about avoiding that, and the unfortunate reality of supported systems where this can not be done.

Any proposals for controlling or changing source or execution encoding should be sent to Study Group 16.

Proposals merely asserting that text is in the execution encoding or translated from the source encoding as currently specified do not need review.

Study Group 16 would like to be made aware of proposals using Unicode encoded literals, but in general would not need to review them.

The assumption that `char`, signed `char`, and unsigned `char`, are in the execution encoding has turned out in practice to be fraught. It may be appropriate to treat data from external sources as having an unknown encoding, rather than the unspecified execution encoding. New facilities may need to provide mechanisms for explicitly specifying the encoding of text.

Formatting

Study Group has already been involved in reviewing `std::fmt`, and will continue as Unicode facilities are added.

Locales

A proposal that uses `std::locale` should be referred to Study Group 16. It is the view of Study Group 16 that large parts of `std::locale` range from unportable to broken in practice. SG16 may be able to provide expertise and experience on correct use of locales.

Providing alternatives is an area of active research, and any new uses of `std::locale` are of interest.

IO

New text input and output proposals should be referred to Study Group 16 to the extent that they expect to deal with text encoding, or want to require a particular encoding. Recent examples include command line arguments, environment, and debugging data.

Using existing input/output facilities, such as `iostreams` or C-style IO does not require review.

Text containers and string builders

New text containers should be sent to Study Group 16 for review, to make sure they can be aligned with Unicode facilities. In particular, there are concerns around breaking encodings by slicing codepoints when operating by code unit, as well as slicing grapheme clusters. String builders may have similar issues, and should also be referred to SG16.

File names

File names present particular challenges, in both host and execution environments. Even using `std::filesystem::path` may present issues if it is expected that the name is displayable. Display names may not roundtrip properly to what the OS APIs require. This is an area of research for Study Group 16, so papers should be referred to SG16.